

Claude Code + Tools and MCPs

Domains 3 (3.1%) and 8 (10.6%) of the CCDV-F blueprint — 13.7% combined.

Sources: Anthropic Partner Academy prep course (Module 3 — Claude Code, MCP & Integration — the primary source for this file, paraphrased from the course rather than reproduced verbatim), cross-checked against [Run Claude Code programmatically](https://code.claude.com/docs/en/headless) (https://code.claude.com/docs/en/headless), [Claude Code settings](https://code.claude.com/docs/en/settings) (https://code.claude.com/docs/en/settings), [Connect Claude Code to tools via MCP](https://code.claude.com/docs/en/mcp) (https://code.claude.com/docs/en/mcp), [Extend Claude with skills](https://code.claude.com/docs/en/skills) (https://code.claude.com/docs/en/skills), [Tool use with Claude](https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview) (https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview), [modelcontextprotocol.io](https://modelcontextprotocol.io/docs/getting-started/intro) (https://modelcontextprotocol.io/docs/getting-started/intro).

Claude Code Operation (3.1%)

The explore → plan → code loop

Claude Code doesn't start writing on receipt of a task — it reads files and traces logic first (**explore**), then produces a structured description of intended edits (**plan**), and only after that plan is understood does it move into **code** (writing/executing changes). This sequencing matters twice: it produces better output (fewer wrong assumptions, more downstream effects caught), and it's *where permission modes plug in* — `plan` mode specifically holds Claude Code in the explore phase, blocking all edits/commands until released.

Permission modes — a risk decision, not a speed decision

Each mode trades oversight for speed differently. Know all six cold — this is dense, specific exam content:

Mode	Auto-approves	Still gated	Notes
<code>default</code>	Reads only	All edits and shell commands	Safe but slow; baseline for unfamiliar codebases
<code>acceptEdits</code>	Reads, file edits, common filesystem commands (<code>mkdir</code> , <code>touch</code> , <code>rm</code> , <code>rmdir</code> , <code>mv</code> , <code>cp</code> , <code>sed</code>) inside the working directory	Everything outside the working directory; protected paths; other shell commands	Trusted local work — but not appropriate if the agent must run arbitrary scripts
<code>plan</code>	Reads only; researches and proposes	All edits/commands until you approve the plan	Exploration/planning on sensitive or unfamiliar code
<code>auto</code>	Everything, but a classifier reviews each action first	Production deploys/migrations, mass deletes, credential exfiltration, force-push to main — blocked by default	Research preview; reduces prompts, doesn't guarantee safety; availability depends on plan/model/admin settings
<code>dontAsk</code>	Only pre-approved allow-listed tools, plus read-only commands	Everything not on the allow list is auto-denied , no confirmation queue	Built for locked-down CI/scripts, not for reducing local friction
<code>bypassPermissions</code>	Everything, no prompts, no safety checks	Only catastrophic commands (<code>rm -rf /</code> , <code>rm -rf ~</code>) still trigger a last-resort prompt	Isolated container/VM only — never on a developer workstation against a live codebase. Also uniquely skips the protected-path guard the other modes keep

Gotcha (real incident pattern from the course): a developer switched to `bypassPermissions` on a "routine" cleanup because prompts felt like friction after three incident-free days. A glob pattern matched files in both the intended `/src/` directory *and* `/deploy/config/prod/`; with no prompt to catch it, production

deployment config got deleted. In `default` or `acceptEdits` mode, the script invocation itself would have prompted before running — `acceptEdits` auto-approves `rm` inside the working directory, but the script call is what would have surfaced first in the safer modes. **Lesson:** `bypassPermissions` silences prompts you didn't anticipate needing, not just the ones you're impatient about — set deny rules on sensitive paths before switching modes, and reach for a classifier-gated mode (`auto`) instead of a full bypass if the goal is just fewer prompts.

Where the human gate belongs

Permission modes and deny rules decide what the agent can do *without asking*. Where a human must still look is a separate question, answered by **worst-case cost**: low-stakes, reversible actions (a formatting fix, an edit confined to the working directory) need no gate — that's what `acceptEdits` is for. Actions that are hard to undo or touch sensitive paths (a write outside the working directory, a destructive shell command) need a gate — enforced deterministically by a deny rule, or surfaced by `default` / `plan` mode. Changes to code your team has marked sensitive should **never** have the agent as the only gate — a human must review before merge regardless of how confident the agent's own review sounds.

Settings scope hierarchy

Scope	File	Applies to
User	<code>~/.claude/settings.json</code>	Every project on the machine — personal preferences that follow you everywhere
Project	<code>.claude/settings.json</code> , committed	Everyone who clones the repo — team-wide conventions, allow/deny rules
Local	<code>.claude/settings.local.json</code> , gitignored	Personal overrides for one project, not shared
Enterprise/Managed	<code>managed-settings.json</code> , admin-set	Cannot be overridden by users or project files — org-wide security controls

A deny rule always wins over an allow rule, regardless of mode. An enterprise-level deny rule is the most durable control available — it can't be removed by any individual developer and applies even under a bypass mode.

CLAUDE.md, rules files, hooks, and subagents — four mechanisms, four different jobs

These solve different problems and shouldn't all be collapsed into one file:

- **CLAUDE.md**: loads in full, every session, unconditionally — the project's always-on memory. `/init` generates a starting version from the codebase; validate before trusting it. **Size is the main failure mode**: a CLAUDE.md that keeps growing dilutes the rules that matter most, since a larger file makes any single instruction a smaller fraction of what loads. Real incident: an 847-line CLAUDE.md contained a correct path restriction (`/legacy/tokens/`) that the agent still violated — not because the rule was missing, but because 846 other lines diluted its weight. **Keep CLAUDE.md to constraints that actually change behavior; move historical/reference content elsewhere, and hold the line around a few hundred lines.**
- **Rules instruction files** (`.claude/rules/`): scoped guidance that loads only when Claude works with matching files, via a `paths` glob in YAML frontmatter (e.g. `paths: ["src/db/**/*.sql"]`). Scoping comes from the frontmatter, **not** from directory placement — a rules file without a `paths` field loads unconditionally, same priority as CLAUDE.md, regardless of which subdirectory it sits in. Use for guidance that's only relevant to one part of the codebase (e.g. "all SQL in the database module needs an explicit transaction boundary") instead of bloating CLAUDE.md with something that's noise everywhere else.

- **Hooks:** run **your own script**, not a model-followed instruction, at a fixed lifecycle point — the difference between a guardrail and a convention. Events: `PreToolUse` (before a tool runs — can inspect and **exit code 2 to block**, with the reason written to stderr as feedback the agent sees), `PostToolUse` (after completion — can't block, right place for auto-formatting/tests/audit logging), `UserPromptSubmit` (inject/validate before the model processes a prompt), `Stop` (end-of-turn cleanup/notifications), `Notification` (fires on a permission request or after 60s idle), `SessionStart` (init state, validate env), `SessionEnd` (teardown, final audit writes). A `PreToolUse` hook blocking a production-config path enforces at every tool call, every session, regardless of permission mode — a CLAUDE.md instruction can be followed inconsistently as the file grows; a hook cannot be skipped.
- **Subagents:** isolated context, no inherited conversation/files/state — see `1_agents_and_workflows.md` for the general mechanics. Course-specific detail: **built-in `Explore` and `Plan` subagents skip CLAUDE.md and git status entirely** (optimized for speed), while `general-purpose` loads both — if a project rule silently doesn't apply to a delegated task, this is often why. Custom subagents don't auto-inherit skills either; a custom subagent needing a specific skill must list it explicitly in its frontmatter.

Skills across four runtimes — the same SKILL.md behaves differently everywhere it runs

A skill authored once can run in Claude Code, the Messages API, the Agent SDK, or Claude Managed Agents — but *where* it loads and *what it can touch* differs sharply per runtime:

Runtime	How it loads	Where steps run	Gotcha
Claude Code	Discovered from <code>.claude/skills</code> on disk, by description match or explicit invocation	Your terminal, local files, under the active permission mode	Filesystem-based, governed by the settings layer
Messages API	Sent with the request, run inside Anthropic's code-execution container (requires code-execution + skills beta headers)	Inside the container, not your machine	A skill assuming local files/tools breaks silently — it isn't running where those files are
Agent SDK	Loaded by the SDK's agent, gated by <code>settingSources / setting_sources</code> — do not rely on a default; set it explicitly	Your environment, once filesystem sources are enabled	Classic surprise: a skill that worked in Claude Code does nothing under the SDK because <code>settingSources</code> was never set, so it never loaded
Managed Agents	Defined once as an API resource listing model/prompt/tools/MCP/skills; loaded server-side, no filesystem discovery	Inside an Anthropic-provisioned sandbox	Public beta (<code>managed-agents-2026-04-01</code> header); sessions stored server-side — not currently eligible for Zero Data Retention or HIPAA BAA coverage

Three portability rules: (1) write the *description* as the matching criterion — a vague description fails to load correctly in every runtime, not just some; (2) don't assume a local filesystem/tools exist in the skill body — a skill that shells out to a local command works in Claude Code and breaks on the Messages API; (3) subagents don't inherit skills in *any* runtime — list them explicitly per-agent if needed.

Custom commands and the skills/commands relationship

In current Claude Code, **skills are the recommended format** for both explicit (`/skill-name`) and automatic (description-match) invocation. The older `.claude/commands/` directory format still works but is legacy. Use `disable-model-invocation: true` in a skill's frontmatter for a workflow that should only ever run when explicitly called, never auto-triggered.

Packaging as a plugin, and the marketplace

A **plugin** bundles skills, hooks, subagents, and MCP servers into one installable unit, distributed via a **marketplace** (a plugin catalog — Anthropic's official one is available by default; add a third-party one hosted on GitHub with `/plugin marketplace add <owner/repo>`). Plugin commands are **automatically**

namespaced by the plugin name (`/payments:run-tests` for a `run-tests` command shipped in a `payments` plugin) — this is why two plugins can ship a same-named command without colliding, and why renaming a plugin renames every command it ships.

Enterprise admins can deploy plugins org-wide through managed settings: a **managed marketplace allowlist** gates which marketplace sources users may add (restricts, doesn't auto-register), paired with `extraKnownMarketplaces` to push a marketplace to all users without requiring the manual add command. Managed-scope deployment sits above user/project settings and can't be overridden.

Gotcha (real incident pattern): a plugin installed cleanly on every teammate's machine, then failed to `run` on all of them except the author's. Root cause: the skill's `SKILL.md` referenced an absolute path (`/Users/alexmorgan/projects/deploy-utils/validate.sh`) that existed only on the author's machine, plus an undocumented environment variable the author had set locally. **Install success and execution success are different things — install just copies files; execution resolves paths/variables against whichever machine is running it.** Fix: use `$CLAUDE_PROJECT_DIR` for scripts stored in the project and `${CLAUDE_PLUGIN_ROOT}` for scripts bundled inside the plugin itself, document every required environment variable, and test the install on a clean machine before distributing.

Session modes

- **Headless / print mode** (`claude -p`): non-interactive, for CI/CD and scripting. Exits `0` on success, non-zero on failure. `--bare` skips auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and `CLAUDE.md` for faster, deterministic CI runs — trades away OAuth/keychain reads, so `ANTHROPIC_API_KEY` must be supplied explicitly.
- **Streaming mode:** `--output-format stream-json` (with `--verbose`, plus `--include-partial-messages` for token-level deltas) emits newline-delimited JSON events, versus `text` (plain) or `json` (single structured result with cost/session metadata).
- **Auto-mode:** `permission_mode: "auto"` — a classifier reviews and approves/denies each action, blocking production deploys/migrations, mass deletes, credential exfiltration, and force-push to main by default; distinct from `bypassPermissions`, which skips evaluation entirely.

```
# CI pattern: bare mode, explicit tool allowlist, structured JSON output
claude --bare -p "Run the test suite and fix any failures" \
  --allowedTools "Bash,Read,Edit" \
  --output-format json
```

Tool Implementation (4.4%)

Tool use / function calling: you define a tool's name, description, and JSON-schema input; Claude decides — based on how well the request matches the tool's *description* — whether to call it. Claude never sees your implementation, only the schema and the result you return. With default `tool_choice: {"type": "auto"}`, Claude calls a tool only when the request maps to its described capability and the answer isn't already in context. This makes **tool description writing** a design task, not documentation — a vague or overlapping description directly causes wrong-tool or no-tool-call failures. **The tool-use loop is not automatic:** Claude only issues the `tool_use` block — your application still has to execute it and return the result; if a schema's selection failures are systematic, the fix belongs in the schema definition itself, not a prompt patch.

Message block structure and the pairing rule that isn't fixable by prompting

A tool-use conversation is built from typed content blocks, and the API enforces a specific structural pairing between them — this is **not** something a prompt tweak can fix, since it's validated before generation even starts:

Block type	Role	Contains	Critical rule
text	Assistant	Claude's prose	Can appear alongside <code>tool_use</code> in the same turn — preserve the full content array, including the text block , when appending that turn to history; dropping it corrupts context for follow-up turns
tool_use	Assistant	Tool name, a unique ID, input arguments	Must be answered by a <code>tool_result</code> in the immediately following user turn, matched by that same ID
tool_result	User	Matching <code>tool_use_id</code> , result content, optional <code>is_error: true</code>	The ID must match the originating <code>tool_use</code> block <i>exactly</i> — this is how Claude connects a result back to its call when one turn issues multiple parallel calls
thinking	Assistant (extended thinking only)	Claude's internal reasoning	Must be passed back to the API unchanged in later turns — the signature verifies it hasn't been edited; any modification (even a summary) breaks the signature and the request is rejected. Redacted thinking blocks follow the same rule despite being encrypted/unreadable.

Missing `tool_result` blocks, IDs that don't match, or turns that arrive out of order all fail request validation — **your code must produce this sequence correctly on every request**, since there's no prompt-level recovery from a structural mismatch.

Schema anatomy: what Claude actually reads to select a tool

Three parts, and the description is what decides selection quality:

- **Name:** specific and short (`get_account_balance`, not `get_data`).
- **Description:** write it in **two parts — when to use it, and when *not* to**. "Use this to find information" gives Claude nothing to distinguish it from any other retrieval tool; "use this to retrieve the current balance for a specific account ID and do not use this for transaction history" gives it an exclusion condition to route on.
- **input_schema:** mark a field `required` only when the call is meaningless without it — marking everything required forces Claude to fabricate values it has no basis for. Leave fields optional when they have sensible defaults or absence itself carries meaning.

Gotcha (real incident pattern): two tools, `search_docs` ("use this to find information about the product") and `get_context_summary` ("use this to retrieve relevant information from the current session") — distinct names, but from Claude's perspective both descriptions say the same thing: "find information." With no exclusion condition, Claude routed to the wrong one on ambiguous inputs, repeatedly. **The fix, once diagnosed, is always the same shape: add one sentence to each description naming when *not* to call it** — e.g. `search_docs`: "...do not call this if the answer is available in the current session context," and `get_context_summary`: "...only use this if the answer is already present in the current session." If descriptions can't be cleanly separated even with exclusion conditions, merge the two tools into one with a `type` parameter instead of continuing to lengthen both descriptions. Note: exclusion conditions that reference "earlier in this conversation" only work if complete history is actually passed on each request — if prior turns get truncated or dropped, Claude can't evaluate the condition and the exclusion logic silently stops working.

Subtask dependency and parallel calls: current models default to issuing multiple independent `tool_use` blocks in one turn when subtasks don't depend on each other. When one tool's output feeds the next call's input, structure it as **separate turns** instead — the second call can't be correctly built until the first result is back. Use `disable_parallel_tool_use` to force strictly one tool call per turn when a workflow requires it.

Dispatch patterns:

- **Client-side:** your application executes the logic and returns the `tool_result` — the model for custom, in-process tools.

- **Server-side/harness dispatch:** the harness (Claude Code, the Agent SDK loop) executes it as part of the loop — built-in tools and MCP tools all dispatch this way; you configure access, not execution.
- **Approval patterns:** auto-approved (`allowed_tools`), gated by a `canUseTool` callback, or denied outright (`disallowed_tools`) — see the permission-mode table above. MCP adds a finer grain: a permission rule can target one tool on a server specifically, `mcp__server__tool` (e.g. an allow rule on `mcp__github__create_issue` while every other GitHub tool still prompts) — **a deny on one tool overrides an allow on the whole server.**

Parallel tool use: multiple `tool_use` blocks in one turn; matching `tool_result` blocks must return together, matched by `tool_use_id`. Read-only tools run concurrently; state-mutating tools (`Edit` , `Write` , `Bash`) run sequentially. A custom tool opts into concurrency via `readOnlyHint` in its annotations.

Tool set construction: keep tool sets small and non-overlapping — every definition costs context on every request, and overlapping descriptions increase the odds of the wrong tool being called. Scope subagents' tools to only what a task needs.

MCP Server Development (2.1%)

What MCP actually solves: without it, if three applications need the same external service, each maintains its own integration (schema + logic, duplicated three times). MCP separates the tool definition from any one application and turns it into a standalone process — an **MCP server** — that any **MCP client** (Claude Code has one built in) can connect to and discover tools from. Build the capability once; every client that connects gets it without re-implementing anything (the exact scenario in the official exam guide's Sample Question 3). From Claude's perspective, a tool discovered via MCP's `ListToolsRequest` handshake is **indistinguishable** from one you registered manually — same description-based routing, same message-block pairing rules; only *who wrote and owns the definition* differs.

Controlling MCP context cost via the API MCP Connector: an `mcp_toolset` object in the `tools` array carries a `default_config` block applied to every tool on a server, overridable per-tool via `configs` keyed by tool name. Two settings matter specifically for context cost: `defer_loading` (delays loading a tool's definition until the model actually needs it — the mechanism behind the deferred-schema behavior described below) and `enabled` (turns individual tools on/off, so you can register a whole server but expose only a subset). Requires the `mcp-client-2025-11-20` beta header — without it, `mcp_toolset` configuration doesn't apply. Note the connector only supports **remote** (HTTP) servers; local stdio servers require Claude Desktop or Claude Code as the client and can't be connected directly through the API.

Three server capabilities, not just tools:

Primitive	What it is	Reach for it when
Tools	Actions the model can call	The model needs to <i>do</i> something
Resources	Read-only data fetched by address and placed directly into context (no tool call) — direct (fixed address) or templated (parameterized address)	Known data should be in context from the start of a turn, and pulling it in directly is cheaper/more predictable than a tool round-trip. <i>Support varies by client — verify before relying on this.</i>
Prompts	Server-exposed, pre-written instruction templates invoked by name	Specific wording matters and every client connecting to the server should get the same vetted instruction, maintained in one place

Transport — matched to where the server runs, not a style preference:

Transport	Use when
stdio	Local process, same machine as the client — personal scripts, dev servers. Cannot be shared across a team.
HTTP	Recommended for anything not local — remote/shared/hosted servers, reached over the network by URL

Transport	Use when
SSE	Legacy — predates HTTP transport, superseded, not recommended for new servers. Treat as historical if you see it in existing config.

Configuration scope — who loads the server, mirrors the settings.json scope pattern:

Scope	Config location	Who gets it
Local	<code>~/.claude.json</code> , per-project entry	Just you, just this project — not committed
User	Personal Claude settings	Just you, across all your projects
Project	<code>.mcp.json</code> , committed to repo root	The whole team, automatically on clone — note : for a stdio server this still spawns a local subprocess per teammate, so each needs the runtime installed (e.g. Node for an <code>npx</code> -launched server)
Enterprise	Managed settings, admin-controlled	Org-wide, pushed without individual configuration

Context cost and tool search: MCP tool schemas are deferred by default, discovered/loaded only when a task calls for them — connecting several servers with large tool sets otherwise costs real context before the agent does anything. An opt-in mode loads definitions upfront when they fit within roughly **10% of the context window**, falling back to deferred loading past that.

Prompt caching, applied to MCP: the same context-cost problem MCP servers create has a caching answer — mark a `cache_control` breakpoint (type `ephemeral`) on the last block you want cached; up to **4 breakpoints per request**; requests process in a fixed order (tools → system prompt → messages), so a breakpoint after tool definitions caches them while keeping messages dynamic. Default TTL is **5 minutes from last read** (resets on each read); opt into a **1-hour** TTL via `ttl: "1h"` on the breakpoint for workloads with longer gaps between requests. Caching only applies above a **minimum token threshold (1,024 tokens on most current models)** — short prompts won't cache even with a breakpoint set. A single changed character before the cache point invalidates it and forces a fresh (paid) write.

Retrieval, briefly (the same context-scaling problem MCP has, generalized): classical RAG pre-builds an embedding index over chunked source material and matches a query's embedding against it at request time; agentic search skips the index and has the model search/fetch live at the moment of need (this is exactly what MCP tool search and Claude.ai Projects' document surfacing both do). Both find a relevant slice and generate from it — the difference is *when* the relevant-slice decision happens (index-build time vs. query time). Retrieval scales because request cost stays flat as the source material grows; it's only as good as what it finds, so poorly-named/organized source material degrades results regardless of which approach you use.

Authentication patterns by service type

Service type	Pattern	Detail
Remote, user identity (SaaS/cloud)	OAuth	Server returns 401, client opens a browser sign-in flow, token issued and stored automatically — no manual credential copying. Right pattern whenever the user's identity is part of authorization (e.g. Linear MCP).
Remote, service identity (internal API)	API key via env variable	Identifies a service account; never committed to config; injected by the runtime (e.g. a CI pipeline secret), not baked into code (e.g. GitHub MCP's personal access token, passed as a header).
Local, filesystem access	No network auth	stdio transport; the security boundary is the filesystem permission model — a deny rule is the governance layer.

Secret handling — three practices, each closing a specific leak path: (1) **separation** — a credential never travels with the config that references it; the file holds a variable reference, the value lives in the environment or a secret store; (2) **where the value lives** — an environment variable for something local/short-lived, a

secret store (centralized, audit-logged, one rotation updates every consumer) for anything shared across services/people; (3) **rotation** — on a schedule and immediately on any suspected exposure; a value baked into committed code can never be cleanly rotated, since old copies persist in history and every hardcoded consumer breaks on change.

Gotcha (real incident pattern): an API key placed inline in `.mcp.json` "temporarily" during setup got committed when the config was shared with the team. Within 48 hours the key existed in four places (local machine, repo history, three teammates' clones, a CI runner). **Overwriting the file in a later commit does not remove the key from history** — rotation was required, and rotation broke two other services configured with the same key. Fix pattern: reference `${WAREHOUSE_MCP_TOKEN}` in the header, never the literal value; back the CLAUDE.md convention ("never write credentials inline to `.mcp.json`") with a `PreToolUse` hook that inspects writes/edits to `.mcp.json` for credential-shaped patterns and blocks them — the instruction communicates intent, the hook enforces it regardless of what the model decides.

Gotcha (OAuth, staging-to-production): an OAuth-authenticated MCP connection that passed every staging test failed entirely in production with a redirect-URI mismatch. **OAuth redirect URIs are registered per host** — the staging registration didn't cover the production hostname, and this wasn't a code defect. Regulated customers often additionally require *separate* OAuth app registrations per environment, not just an added redirect URI. Put the registration step in the deployment checklist, not the incident log.

What regulated/enterprise deployment adds on top of "it works": identity auditability (who is the model acting as), data residency (where does data leave the org — an HTTP endpoint pinned to a specific region plus a matching platform deployment gives a checkable answer), configuration lock (enterprise managed settings an individual developer can't override), and access logging (a `PostToolUse` hook logging every tool call to an audit store — fires deterministically regardless of model behavior, unlike a log the model could theoretically skip).

Agentic Customization (4.1%): built-in Tools vs. custom Tools vs. Skills vs. MCP

The "which do I reach for" decision the exam blueprint calls out by name:

	Built-in tools	Custom tools	Skills	MCP servers
What it is	Pre-built capabilities shipped with Claude Code/Agent SDK	Functions <i>you</i> define with a schema, for one specific app/integration	Portable procedural knowledge (<code>SKILL.md</code>) Claude reads on demand	A reusable, independently-deployed server exposing resources/tools/prompts
Reach for it when	The task is generic and the built-in already does it	One app needs one specific internal function, no reuse elsewhere	A repeatable process/judgment call, no new live data access needed	Multiple apps need the same live data/action, maintained independently
Not the right fit when	Custom business logic or a proprietary integration is needed	The same capability needs sharing across unrelated apps	The task is really about fetching live external data	Overkill for a single, app-specific, one-off function

MCP connects Claude to data; Skills teach Claude what to do with that data. A Skill saying "look up the latest deployment status" cannot itself reach the deployment system — it still needs an MCP server (or custom tool) underneath it. Built-in tools are always available regardless of what MCP/Skills you add — the layers are additive, never substitutive. A typical production setup uses all three together: MCP servers for live external systems, custom tools for app-specific one-off functions, Skills to encode the judgment calls and reusable procedures tying them together — and, per the packaging section above, a **plugin** to bundle and distribute that whole combination as one installable, versioned unit.